LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Nabilah Nur Azizah NIM. 2510817120015

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nabilah Nur Azizah
NIM : 2510817120015

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN i

DAFTAR ISI ii

DAFTAR GAMBAR iii

DAFTAR TABEL iv

SOAL 1 5

A. Source Code 5

B. Pembahasan 6

SOAL 2 8

A. Source Code 8

B. Output Program 11

C. Pembahasan 11

SOAL 3 14

A. Source Code 14

B. Pembahasan 16

SOAL 4 18

A. Source Code 18

B. Output Program 20

C. Pembahasan 20

DAFTAR GAMBAR

Gambar 1. Screenshot hasil jawaban soal 2 11
Gambar 2. Screenshot hasil jawaban soal 4 20

DAFTAR TABEL

Tabel 1. Source Code stack.cpp 5
Tabel 2. Source Code prob1.cpp 8
Tabel 3. Source Code queue.cpp 14
Tabel 4. Source Code prob2.cpp 18

SOAL 1

A. Source Code

Tabel 1. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
            throw  std::overflow_error("Stack  penuh
(overflow)");
    }

    if (isEmpty(s)) {
        s->top = &s->data[0];
    } else {
        s->top++;
    }
```

```
26
27      *(s->top) = value;
28  }
```

B. Pembahasan

Pada baris [1 dan 2], syntax `#include "line.h"` digunakan untuk mengimpor semua definisi dari file header, seperti struct Stack dan nilai MAX. `#include <stdexcept>` digunakan untuk mengimpor Pustaka standar C++ yang berisi jenis-jenis eksepsi (kesalahan) siap pakai. Dengan menggunakan kelas-kelas ini, kesalahan dapat dideteksi dan dijelaskan lebih spesifik menggunakan pesan kesalahan saat program berjalan.

Pada baris [4-6], syntax `void init(Stack* s)` digunakan untuk menginisialisasi stack agar berada dalam kondisi kosong di awal program dan digunakan untuk membuka fungsi `init` yang menerima pointer `s` ke objek Stack. Syntax `s->top = nullptr` berfungsi untuk mengakses anggota top dari struct Stack yang ditunjuk oleh `s` menggunakan operator →. Nilainya akan diisi dengan `nullptr` (kosong/tidak menunjuk ke memori mana pun). Ini adalah tanda awal bahwa Stack berada dalam keadaan kosong.

Pada baris [8-10], syntax `bool isEmpty(const Stack* s)` digunakan untuk memeriksa apakah stack dalam keadaan kosong, fungsi ini akan membuka fungsi yang mengembalikan nilai Benar/Salah (`bool`). Sedangkan syntax `return s->top == nullptr;` berfungsi untuk mengevaluasi sebuah kondisi yaitu "Apakah pointer `top` saat ini bernilai `nullptr`?". Jika ya, kembalikan `true` (Stack kosong). Jika tidak, kembalikan `false` (Stack ada isinya).

Pada baris [12-14], syntax `bool isFull(const Stack* s)` berfungsi untuk memeriksa apakah kapasitas stack sudah terpakai maksimal menggunakan fungsi `return s->top == &(s->data[MAX - 1]);` untuk

membandingkan alamat memori yang sedang ditunjuk oleh `top` saat ini dengan alamat memori (`&`) elemen terakhir di dalam array `data` pada indeks terakhir `(MAX - 1)`. Jika MAX adalah 100, indeks terakhirnya adalah 99. Jika sama, berarti array sudah terisi sampai ujung lalu kembalikan `true` (Stack penuh).

Pada baris [16-19], syntax `void push(Stack* s, int value)` berfungsi untuk menyisipkan elemen `value` ke posisi teratas Stack. `if (isFull(s))` untuk mengecek kondisi apakah Stack sudah penuh. `throw std::overflow_error("...");` baris ini akan menghentikan paksa eksekusi dan melempar pesan *error* "overflow" jika Stack penuh dan mencegah program menulis data di luar batas memori array yang bisa berakibat fatal.

Pada baris [21-25], syntax `if (isEmpty(s))` berfungsi untuk mengecek apakah ini adalah data pertama yang dimasukkan (karena stack masih kosong). `s->top = &s->data[0];` jika ini elemen pertama, arahkan pointer `top` menuju alamat memori (`&`) dari array elemen pertama (indeks ke-0). `} else {` Jika Stack sudah memiliki isi sebelumnya (tidak kosong)... `s->top++;` ...maka geser posisi pointer `top` naik 1 tingkat. Dalam bahasa mesin, ini menggeser pointer memori untuk menunjuk ke "kotak" array berikutnya yang masih kosong.

Pada baris [27 dan 28], syntax `*(s->top) = value;` tanda bintang `*` (*dereference*) berarti "Buka kotak memori yang saat ini ditunjuk oleh pointer `top`". Lalu isi kotak tersebut dengan angka dari variabel `value`.

SOAL 2

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

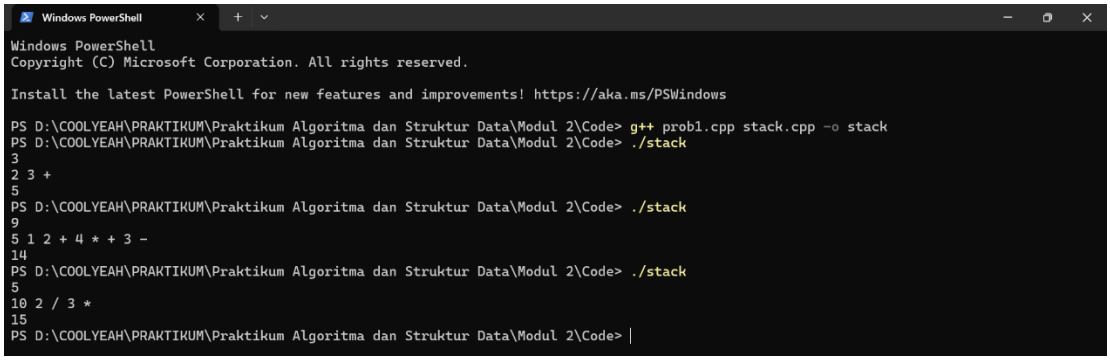
Tabel 2. Source Code prob1.cpp

```
#include <iostream>
#include <sstream>
#include <string>
#include "stack.h"

using namespace std;

int main() {
    Stack s;
    init(&s);

    int n;
    cin >> n;
    cin.ignore();

    string line;
    getline(cin, line);
```

```

    stringstream ss(line);
    string token;

    try {
        while (ss >> token) {
            if (token == "+" || token == "-" || token == "*" || token == "/") {
                if (isEmpty(&s)) throw runtime_error("Operand kurang");
                int b = peek(&s); pop(&s);

                if (isEmpty(&s)) throw runtime_error("Operand kurang");
                int a = peek(&s); pop(&s);

                int result;

                if (token == "+") result = a + b;
                else if (token == "-") result = a - b;
                else if (token == "*") result = a * b;
                else {
                    if (b == 0) throw runtime_error("Pembagian nol");
                    result = a / b;
                }

                push(&s, result);
```

```
            }
            else {
                int num = stoi(token);
                push(&s, num);
            }
        }

        if (isEmpty(&s)) throw runtime_error("Stack kosong");

        int hasil = peek(&s);
        pop(&s);

        if (!isEmpty(&s)) throw runtime_error("Ekspresi tidak valid");

        cout << hasil << endl;

    } catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }

    return 0;
}
```

B. Output Program



Gambar 1. Screenshot hasil jawaban soal 2

C. Pembahasan

Pada baris [1-6], syntax yang didepan penulisannya menggunakan simbol pagar "#" adalah syntax untuk mengakses library standar C++. `#include <iostream>` adalah pustaka standar untuk input/output (seperti `cin` dan `cout`). `#include <sstream>` adalah pustaka untuk string stream dan sangat berguna untuk memecah teks Panjang menjadi kata-kata (token) berdasarkan spasi. `#include <string>` adalah pustaka untuk menggunakan tipe data teks `string`. Sedangkan `#include "stack.h"` adalah header dari kerangka struktur Stack. `using namespace std;` adalah fungsi yang memungkinkan penulisan `cout`, `cin`, dan `string` tanpa perlu menambahkan awalan `std::` terus-menerus.

Pada baris [8-10], syntax `int main() {` adalah titik awal eksekusi program. `Stack s;` membuat sebuah objek/variabel `s` bertipe `Stack`. `init(&s);` berfungsi untuk memanggil fungsi `init` dengan mengirimkan alamat memori dari `s` (`&s`) untuk memastikan *pointer* `top` bernilai `nullptr` (Stack siap dan kosong).

Pada baris [12-14], syntax `int n; cin >> n;` digunakan untuk mendeklarasikan variabel `n` dan membaca input angka dari pengguna. Biasanya ini ditujukan untuk membaca "berapa banyak elemen/token yang akan diinput".

`cin.ignore();` adalah baris yang penting, biasanya setelah menekan 'Enter' saat memasukkan angka `n`, karakter 'Enter' (newline `\n`) masih tertinggal di dalam *buffer*. Fungsi ini bertugas membersihkan/membuang sisa *newline* tersebut agar tidak mengganggu proses pembacaan teks selanjutnya.

Pada baris [16-20], syntax `string line; getline(cin, line);` digunakan untuk membaca satu baris teks penuh (termasuk spasi) yang diketik pengguna dan menyimpannya ke variabel `line`. Selanjutnya, `stringstream ss(line);` fungsinya untuk memasukkan `line` ke dalam mesin `stringstream` bernama `ss`. Mesin ini akan memperlakukan string tersebut seperti aliran input biasa (`cin`) yang otomatis memotong teks jika bertemu spasi. `string token;` adalah variabel penampung sementara untuk setiap potongan teks (kata/angka) yang keluar dari `ss`.

Pada baris [22], syntax `try {` digunakan untuk membuka blok penanganan eksepsi. Jika ada *error* (seperti Stack kosong, pembagian dengan nol), program tidak akan langsung *crash*, melainkan akan ditangkap oleh blok `catch` di bagian bawah.

Pada baris [23], syntax `while (ss >> token) {` digunakan untuk mengambil satu potongan teks (dipisahkan spasi) dari `ss` dan memasukkannya ke variabel `token`. Proses ini diulang (loop) selama masih ada kata yang tersisa di kalimat.

Pada baris [24-29], terdapat logika operator yang akan digunakan program. Jika teks yang dibaca adalah operator matematika, maka harus mengambil dua angka teratas dari stack. Validasi dan Ambil `b` akan mengecek apakah stack kosong. Jika aman, intip angka teratas (`peek`), simpan di variabel `b`, lalu buang dari stack (`pop`). Elemen yang ada di-pop pertama kali selalu menjadi operand kanan (`b`). Validasi dan Ambil `a` akan melakukan hal yang sama untuk mengambil angka kedua teratas, disimpan di variabel `a` (operand kiri). Jika stack sudah kosong sebelum `a` diambil, program melempar *error* "Operand kurang".

Pada baris [31-42], terdapat proses perhitungan matematika yang melakukan perhitungan berdasarkan tipe operatornya. Urutannya harus `a` (operator) `b`, bukan sebaliknya. Khusus untuk pembagian (`/`), jika nilai `b` adalah `0`, lempar *error*. Membagi dengan nol dalam C++ akan memicu *fatal crash*. Selanjutnya, hasil perhitungan (`result`) dimasukkan kembali ke dalam Stack (`push`) untuk menjadi operand bagi operator selanjutnya (jika ada).

Pada baris [43-47], syntax `else {` berarti, jika teks bukan operator, maka teks tersebut dianggap sebagai angka. `int num = stoi(token);` fungsinya untuk mengubah (`parsing`) teks string menjadi angka integer menggunakan fungsi *String-to-Integer* (`stoi`). Lalu syntax `push(&s, num);` digunakan untuk memasukkan angka tersebut ke dalam tumpukan.

Pada baris [48-56], ditampilkan validasi akhir dari perhitungan dan output. Setelah loop selesai (semua token terbaca), ekspresi RPN yang valid hanya boleh menyisakan tepat 1 angka di dalam Stack. Angka tersebut adalah hasil akhir. `if (isEmpty(&s)` untuk mengecek jika setelah loop ternyata stack tidak punya apa-apa (mungkin user menginput teks kosong). `int hasil...` akan mengambil nilai akhir tersebut dan membuangnya dari stack. Selanjutnya, `if (!isEmpty(&s))` akan mengecek apakah *masih* ada angka lain yang tersisa di tumpukan setelah hasil akhir diambil. Jika ada, berarti pengguna memasukkan terlalu banyak angka tanpa operator yang cukup. `cout << hasil...` akan mencetak hasil akhir ke layar.

Pada baris [58-63], syntax `catch (const exception& e) {` berarti, jika di dalam blok `try` ada baris kode yang menjalankan perintah `throw` (seperti *stack overflow*, *underflow*, atau pembagian nol), program akan "melompat" langsung ke sini. `e.what()` akan mengambil pesan teks yang tadi disematkan saat *error* dilempar (misalnya: "Operand kurang" atau pesan dari fungsi push/pop di `stack.cpp`) dan mencetaknya ke layar dengan rapi. Program berakhir dengan baik tanpa crash sistem.

SOAL 3

A. Source Code

Tabel 3. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return (q->front == nullptr);
}

bool isFull(const Queue* q) {
    if (q->front == nullptr) return false;

    return (q->rear == &(q->data[MAX - 1]));
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::overflow_error("Queue penuh!");
    }

    if (isEmpty(q)) {
        q->front = q->data;
        q->rear = q->data;
```

```
    } else {
        q->rear++;
    }

    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong!");
    }

    if (q->front == q->rear) {
        q->front = nullptr;
        q->rear = nullptr;
    } else {
        q->front++;
    }
}

int front(const Queue* q) {
    if (isEmpty(q)) {
        throw std::underflow_error("Queue kosong!");
    }

    return *(q->front);
}

int back(const Queue* q) {
    if (isEmpty(q)) {
```

```
57          throw std::underflow_error("Queue kosong!");
58      }
59
60      return *(q->rear);
61  }
```

B. Pembahasan

Pada baris [1 dan 2], syntax `#include "queue.h"` akan menghubungkan program ini dengan file `queue.h` yang sudah dibuat sebelumnya. `#include <stdexcept>` memuat pustaka standar C++ untuk menggunakan fitur pelemparan error bawaan (seperti *overflow* dan *underflow*).

Pada baris [4-7], terdapat inisialisasi `void init(Queue* q)`. Tujuannya untuk mengatur kondisi awal Queue yang baru dibuat. Karena antrean masih kosong, pointer `front` dan `rear` diatur agar menunjuk ke "hampa" (`nullptr` atau nol pointer). Tanda → digunakan untuk mengakses anggota struct karena `q` adalah sebuah pointer.

Pada baris [9-11], syntax `bool isEmpty(const Queue* q)` berfungsi untuk mengecek apakah antrean kosong. Jika pointer `front` tidak menunjuk ke alamat memori manapun (`nullptr`), berarti tidak ada data di antrean. Selain itu juga berfungsi untuk mengembalikan nilai `true` jika ya, dan `false` jika tidak.

Pada baris [13-17], syntax `bool isFull(const Queue* q)` digunakan untuk mengecek apakah antrean sudah penuh kapasitasnya. Baris pertama memastikan bahwa jika antrean kosong (`front == nullptr`), pasti belum penuh. Baris kedua melakukan pengecekan alamat memori, `&(q->data[MAX - 1])` akan mengambil alamat memori dari elemen terakhir di dalam array. Jika pointer `rear` menunjuk persis ke alamat ini, berarti antrean sudah mencapai batasnya.

Pada baris [19-22], syntax `void enqueue(Queue* q, int value)` digunakan untuk mengecek kapasitas. `if (isFull(q))` Jika penuh, `throw std::overflow_error("Queue penuh!")` program akan melemparkan error yang menghentikan eksekusi sambil memberikan pesan "Queue penuh!".

Pada baris [24-32], terdapat fungsi yaitu, Jika kosong maka pointer `front` dan `rear` langsung diarahkan ke alamat awal array (`q->data` identik dengan `&(q->data[0])`). Jika tidak kosong maka memajukan posisi penunjuk belakang. `q->rear++` artinya menggeser pointer satu langkah ke kanan (ke alamat memori memori elemen berikutnya di array). `*(q->rear) = value;` akan menyimpan angka yang dimasukkan (`value`) tepat di alamat memori yang sedang ditunjuk oleh pointer `rear`.

Pada baris [34-37], syntax `void dequeue(Queue* q)` digunakan untuk mengeluarkan data. Jika antrean kosong namun pengguna memaksa mengambil data, program akan melempar *error*.

Pada baris [39-45], Jika `front` dan `rear` menunjuk ke alamat yang sama, artinya hanya ada satu elemen tersisa. Jika elemen itu dikeluarkan, antrean kembali kosong (semua di-reset ke `nullptr`). Jika ada lebih dari satu elemen, cukup majukan posisi antrean terdepan dengan `q->front++`. Data lama di dalam array secara fisik tidak dihapus, kita hanya mengabaikannya dengan menggeser pintu depan (`front`) ke elemen berikutnya.

Pada baris [47-61], syntax `int front(const Queue* q) { ... return *(q->front); }` dan `int back(const Queue* q) { ... return *(q->rear); }` digunakan untuk melihat elemen ujung, yaitu `front` dan `back`. Keduanya sama-sama melakukan validasi kosong terlebih dahulu. Jika aman, operator dereferensi (`*`) digunakan untuk mengambil isi dari alamat memori yang ditunjuk oleh `front` (untuk melihat antrean terdepan) atau `rear` (untuk melihat antrean paling belakang) tanpa mengeluarkannya dari array.

SOAL 4

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

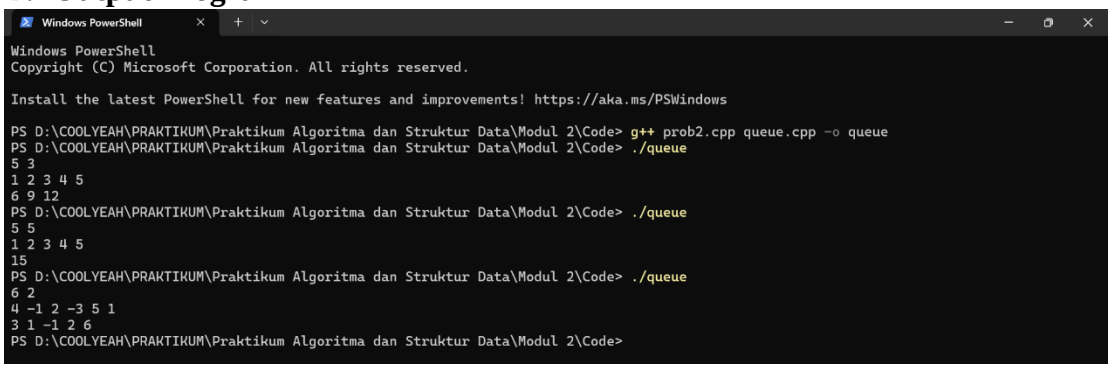
A. Source Code

Tabel 4. Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int arr[100];

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    if (k <= 0 || k > n) {
        cout << "Invalid k" << endl;
        return 0;
    }
```

```

    Queue q;
    init(&q);

    int sum = 0;

    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        sum += arr[i];
    }

    cout << sum;

    for (int i = k; i < n; i++) {
        sum -= front(&q);
        dequeue(&q);

        enqueue(&q, arr[i]);
        sum += arr[i];

        cout << " " << sum;
    }

    cout << endl;

    return 0;
}
```

B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 2\Code> g++ prob2.cpp queue.cpp -o queue
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 2\Code> ./queue
5 3
1 2 3 4 5
6 9 12
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 2\Code> ./queue
5 5
1 2 3 4 5
15
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 2\Code> ./queue
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 2\Code>
```

Gambar 2. Screenshot hasil jawaban soal 4

C. Pembahasan

Pada baris [1-3], syntax `#include <iostream>` digunakan untuk memuat pustaka `iostream` untuk operasi input/output standar C++. Baris `#include "queue.h"` memanggil struktur Queue yang telah dibuat sebelumnya program ini. `using namespace std;` agar kita tidak perlu menulis `std::` berulang kali.

Pada baris [5-7], syntax `int main() {` adalah titik awal eksekusi program. `int n, k; cin >> n >> k;` untuk mendeklarasikan dan menerima dua input angka. `n` adalah total banyak angka yang akan dimasukkan, `k` adalah ukuran windows (jumlah elemen berurutan yang akan dihitung totalnya dalam satu waktu).

Pada baris [9-13], syntax `int arr[100];` membuat array `arr` dengan kapasitas maksimal 100 untuk menyimpan angka-angka utama. Looping `for` digunakan untuk membaca dan memasukkan `n` buah angka dari pengguna ke dalam array tersebut.

Pada baris [15-18], syntax `if (k <= 0 || k > n)` digunakan untuk validasi keamanan. Ukuran window (`k`) tidak boleh kurang dari atau sama dengan 0, dan tidak boleh lebih besar dari total elemen (`n`). Jika tidak valid, program mencetak pesan error dan langsung berhenti (`return 0;`).

Pada baris [20 dan 21], syntax `Queue q;` dan `init(&q);` adalah inisialisasi dan jendela pertama (fFrst Window) yang menghitung total dari kelompok angka pertama berukuran `k`. Fungsi ini membuat sebuah variabel antrean bernama `q` berdasarkan `struct Queue` dari file header. Fungsi `init(&q)` dipanggil untuk mengatur pointer `front` dan `rear` menjadi kosong (`nullptr`). Tanda `&` digunakan untuk mengirimkan alamat memori dari `q` (karena fungsi `init` meminta pointer).

Pada baris [23-30] * mendeklarasikan `sum` untuk menyimpan total penjumlahan. Looping berjalan hanya untuk `k` elemen pertama. Di setiap iterasi, angka dimasukkan ke dalam antrean (`enqueue`) dan nilainya ditambahkan ke variabel `sum`. Setelah loop selesai, `sum` dari kelompok pertama dicetak ke layar. Ini adalah titik awal dari window.

Pada baris [32-34], terjadi proses sliding window (jendela bergeser), ini adalah inti dari algoritma. Daripada menghitung ulang `k` angka dari awal setiap kali jendela bergeser, program ini menggunakan metode yang jauh lebih efisien. syntax `for (int i = k; i < n; i++) {` akan memulai Looping dari indeks `k` (elemen tepat setelah kelompok pertama) hingga elemen terakhir `n`. Kita mulai menggeser jendela ke kanan satu per satu. Pada syntax `sum -= front(&q);` dan `dequeue(&q);` saat jendela bergeser ke kanan, elemen paling kiri dari jendela sebelumnya harus ditinggalkan. `front(&q)` melihat elemen paling depan di antrean (elemen paling tua). Nilainya dikurangkan dari `sum`. Sedangkan, `dequeue(&q)` secara resmi "membuang" elemen tua tersebut dari antrean.

Pada baris [36 dan 37], baris ini digunakan untuk memasukkan data terbaru setelah membuang yang lama, `enqueue(&q, arr[i])` memasukkan angka baru ke bagian belakang antrean. `sum += arr[i]` menambahkan nilai angka baru tersebut ke total penjumlahan.

Pada baris [39 dan 40], syntax cout << " " << sum; } digunakan untuk mencetak spasi dan nilai `sum` terbaru dari jendela yang baru saja bergeser.

Pada baris [42-45], syntax `cout << endl;` akan mencetak baris baru (enter) untuk kerapian dab mengakhiri program dengan sukses `return 0;`.